

COMP4131: Data Modelling and Analysis

Lecture 6: Linear Regression and Logistic Regression

Daokun Zhang

University of Nottingham Ningbo China

daokun.zhang@nottingham.edu.cn

March 30, 2026

Overview

1 Linear Regression

2 Logistic Regression

Linear Regression

A Simple Example of Linear Regression

这表示 y 和 x 之间是线性关系： x 每增加 1, y 就变化 k
如果 $k > 0$, 说明正相关；如果 $k < 0$, 说明负相关。

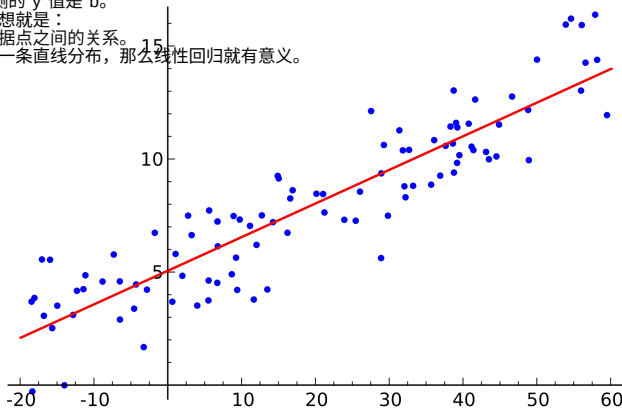
截距 b 的意思

当 $x=0$ 时, 模型预测的 y 值是 b 。

线性回归最基本的思想就是：

用一条直线去近似数据点之间的关系。

如果数据点大致沿着一条直线分布, 那么线性回归就有意义。



$$y = kx + b$$

x : 输入变量
 y : 输出变量
 k : 斜率
 b : 截距

Linear Regression with Multiple Input Variables

Given the ^{输入特征向量} input sample $\mathbf{x} = (x_1, x_2, \dots, x_D)^T$ with D feature dimensions, we can predict the target value $y \in \mathbb{R}$ as

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1 x_1 + \dots + w_D x_D,$$

where ^{参数向量} $\mathbf{w} = (w_0, w_1, \dots, w_D)^T$ are the parameters.

Here ^{预测输出} $y(\mathbf{x}, \mathbf{w})$ is

- a linear function of the parameters $w_0, w_1, \dots, w_D$, and
- a linear function of the input variables $x_1, x_2, \dots, x_D$.

比如预测房价：

$$\text{price} = w_0 + w_1() + w_2() + w_3()$$

这表示房价由多个因素线性叠加决定。

Linear Basis Function Models

We can extend the linear regression model to the linear basis function models by considering **linear combinations of fixed nonlinear functions of the input variables**, of the form

我们可以通过考虑输入变量固定非线性函数的线性组合形式，将线性回归模型扩展至线性基函数模型。

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}),$$

where $\phi_j(\mathbf{x})$ are the basis functions, and M is the number of parameters.

To be more concise, we define an additional dummy '**basis function**'

$\phi_0(\mathbf{x}) = 1$ for the **bias parameter** w_0 so that

模型虽然对参数 w 是线性的，但 $\phi(\mathbf{x})$ 本身可以是非线性的。
也就是说：对 w 线性对 x 不一定线性

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

- $\phi_j(x)$: 第 j 个基函数
- w_j : 对应的权重
- $\boldsymbol{\phi}(x)$: 所有基函数组成的向量

where $\mathbf{w} = (w_0, \dots, w_{M-1})^T$ and $\boldsymbol{\phi}(\mathbf{x}) = [\phi_0(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x})]^T$.

Examples of Basis Functions

填补/删除缺失值, 特征归一化

- Feature Pre-processing: impute/remove missing values, feature normalization

- Feature Selection 特征选择

- Dimension Reduction 降维

恒等基函数

- Identity Basis Function: $\phi(\mathbf{x}) = \mathbf{x}$ 就是不做变换, 直接使用原始输入

- Polynomial Basis Function: $\phi_j(x) = x^j$ 可以拟合多项式曲线: 例如行, x, x^2, x^3
多项式基函数

- Gaussian Basis Function: $\phi_j(x) = \exp\left\{-\frac{(x-\mu_j)^2}{2s^2}\right\}$
高斯基函数

• μ_j : 中心

• s : 宽度

含义

离中心 μ_j 越近, 函数值越大; 越远越小。

- Sigmoidal Basis Function: $\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right)$ with $\sigma(a) = \frac{1}{1+\exp(-a)}$

输入平滑地映射到 0 和 1 之间。

- Fourier Basis Function

通常用于周期型数据, 比如声音信号、季节变化。

- ...

Maximum Likelihood and Least Squares

Now consider a data set of inputs $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ with corresponding target values $\mathbf{t} = \{t_1, \dots, t_N\}$, to find the best parameter $\mathbf{w}$, we need to minimize the sum-of-squares error function:

- t_n : 第 n 个样本的真实值
- $y(x_n, w)$: 模型预测值
- $t_n - y(x_n, w)$: 预测误差

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - y(\mathbf{x}_n, \mathbf{w})\}^2$$

为什么平方

- 避免正负抵消
- 大误差惩罚更重
- 方便求导优化

$$= \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2.$$

这页核心

平方误差是线性回归的标准损失函数。

Question: Why the square error is used to design the objective function?

Maximum Likelihood and Least Squares

We assume that the target variable t is given by a deterministic function $y(\mathbf{x}, \mathbf{w})$ with additive Gaussian noise so that

$$t = y(\mathbf{x}, \mathbf{w}) + \epsilon,$$

where ϵ is a zero mean Gaussian random variable with precision (inverse variance) β . Thus we can write

$$p(t|\mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \beta^{-1}),$$

where

$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp\left\{-\frac{1}{2\sigma^2}(x - \mu)^2\right\}.$$

Maximum Likelihood and Least Squares

For the given data set $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and $\mathbf{t} = \{t_1, \dots, t_N\}$, the joint likelihood function can be expressed as

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n | \mathbf{w}^T \phi(\mathbf{x}_n), \beta^{-1}).$$

最大化对数似然，等价于最小化平方误差。

Taking the logarithm of the likelihood function, and making use of the standard form for the univariate Gaussian, we have

$$\begin{aligned} \ln p(\mathbf{t}|\mathbf{w}, \beta) &= \sum_{n=1}^N \ln \mathcal{N}(t_n | \mathbf{w}^T \phi(\mathbf{x}_n), \beta^{-1}) \\ &= \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi) - \frac{\beta}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 \\ &= \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi) - \beta E_D(\mathbf{w}). \end{aligned}$$

Maximum Likelihood and Least Squares

The 损失函数的梯度 gradient of the sum-of-squares error function $E_D(\mathbf{w})$ takes the form

$$\nabla E_D(\mathbf{w}) = - \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\} \phi(\mathbf{x}_n)^T.$$

Setting this gradient to zero gives

$$0 = - \sum_{n=1}^N t_n \phi(\mathbf{x}_n)^T + \mathbf{w}^T \left(\sum_{n=1}^N \phi(\mathbf{x}_n) \phi(\mathbf{x}_n)^T \right).$$

Solving for $\mathbf{w}$, we obtain

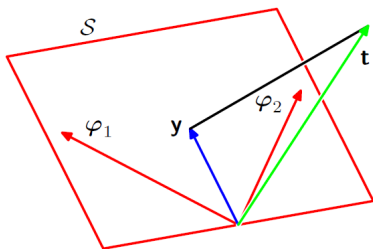
$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t},$$

which are known as the *normal equations* for the least squares problem.

Here $\Phi = [\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_N)]^T$ is an $N \times M$ matrix.

Geometry Interpretation of Least Squares

最小二乘就是：在模型能表示的子空间中，找到离真实目标 $\mathbf{t}$ 最近的那个点。
也就是把 $\mathbf{t}$ 投影到子空间上。



- consider $\mathbf{t} = \{t_1, \dots, t_N\}$ is a vector in an N -dimensional space
- define $\varphi_j = \{\phi_j(\mathbf{x}_1), \dots, \phi_j(\mathbf{x}_N)\}$ as the j th column vector of Φ
- $\{\varphi_0, \dots, \varphi_{M-1}\}$ span a subspace S of dimensionality M , if $M < N$
- the least-squares solution corresponds to the choice of $\mathbf{y} = \{y(\mathbf{x}_1, w), \dots, y(\mathbf{x}_N, w)\}$ that lies in subspace S and that is closest to $\mathbf{t}$, i.e, the projection of $\mathbf{t}$ onto S

Regularized Least Squares

To avoid over-fitting, we consider adding a regularization term to an error function, so that the total error function to be minimized takes the form

$$E(\mathbf{w}) = E_D(\mathbf{w}) + \lambda E_W(\mathbf{w}),$$

where λ is the regularization coefficient, $E_D(\mathbf{w})$ is the data-dependent error, and $E_W(\mathbf{w})$ is the regularization term:

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2,$$
$$E_W(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}.$$

Then the total error function becomes

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}.$$

Regularized Least Squares

The gradient of the new error function $E(\mathbf{w})$ takes the form

$$\nabla E(\mathbf{w}) = - \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\} \phi(\mathbf{x}_n)^T + \lambda \mathbf{w}.$$

Setting this gradient to zero gives

$$0 = - \sum_{n=1}^N t_n \phi(\mathbf{x}_n)^T + \mathbf{w}^T \left(\sum_{n=1}^N \phi(\mathbf{x}_n) \phi(\mathbf{x}_n)^T \right) + \lambda \mathbf{w}^T.$$

Solving for $\mathbf{w}$, we obtain

$$\mathbf{w}^* = (\lambda \mathbf{I} + \Phi^T \Phi)^{-1} \Phi^T \mathbf{t},$$

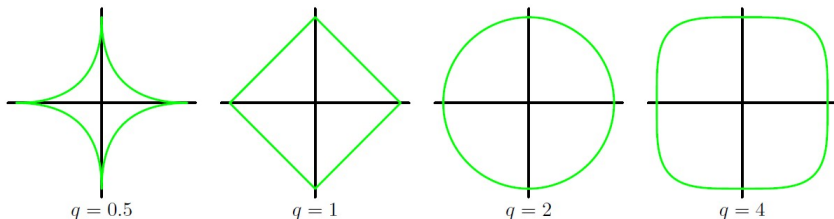
where $\mathbf{I}$ is the $M \times M$ identity matrix.

Regularized Least Squares

A more general regularizer is sometimes used, for which the regularized error takes the form

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \sum_{j=1}^M |w_j|^q,$$

where $q = 2$ corresponds to the *quadratic* regularizer, and the case of $q = 1$ know as the *lasso* regularizer.

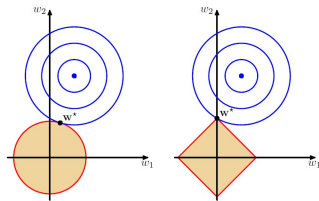


Contours of the regularization term for various values of the parameter q .

Regularized Least Squares

Minimizing the regularized error $E(\mathbf{w})$ is equivalent to

$$\min_{\mathbf{w}} \underbrace{\frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2}_{E_D(\mathbf{w})}, \text{ s.t. } \sum_{j=1}^M |w_j|^q \leq \eta.$$



The contours of the unregularized error function (blue) along with the constraint region for the quadratic regularizer $q = 2$ on the left and the lasso regularizer $q = 1$ on the right, in which the optimum value for the parameter vector $\mathbf{w}$ is denoted by $\mathbf{w}^*$. The lasso gives a sparse solution in which $w_1^* = 0$.

The Bias-Variance Decomposition

Given the joint distribution $p(\mathbf{x}, t)$ of the input $\mathbf{x}$ and target value t , we can determine the specific estimate $y(\mathbf{x})$ of the value t for each input $\mathbf{x}$ (a mapping function from $\mathbf{x}$ to t), as the function that minimizes the expected squared loss

$$\mathbb{E}_{(\mathbf{x}, t)}[L] = \iint \{y(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt.$$

The optimal $y(\mathbf{x})$ should be

$$y^*(\mathbf{x}) = h(\mathbf{x}) = \int t p(t|\mathbf{x}) dt = \mathbb{E}_t[t|\mathbf{x}].$$

For a given estimate function $y(\mathbf{x})$, the expected squared loss can be expressed as

$$\begin{aligned} \mathbb{E}_{(\mathbf{x}, t)}[L] &= \iint \{y(\mathbf{x}) - h(\mathbf{x}) + h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt \\ &= \int \{y(\mathbf{x}) - h(\mathbf{x})\}^2 p(\mathbf{x}) d\mathbf{x} + \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt. \end{aligned}$$

The Bias-Variance Decomposition

In general, the prediction function $y(\mathbf{x})$ is learned from a given data set $\mathcal{D}$, then we use $y(\mathbf{x}; \mathcal{D})$ to denote $y(\mathbf{x})$, and write the squared difference between $y(\mathbf{x})$ and $h(\mathbf{x})$ as

$$\{y(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2.$$

If we add and subtract the quantity $\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]$ inside the braces, and then expand, we obtain

$$\begin{aligned} & \{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] + \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2 \\ &= \{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]\}^2 + \{\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2 \\ & \quad + 2 \{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]\} \{\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}. \end{aligned}$$

We now take the expectation of this expression with respect to $\mathcal{D}$ and note that the final term will vanish, giving

The Bias-Variance Decomposition

$$\begin{aligned} \mathbb{E}_{\mathcal{D}} [\{y(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2] \\ = \underbrace{\{\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2}_{(\text{bias})^2} + \underbrace{\mathbb{E}_{\mathcal{D}} [\{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]\}^2]}_{\text{variance}} \end{aligned}$$

So far, we have considered a single input value $\mathbf{x}$. If we substitute this expansion back into the expected loss

$$\begin{aligned} \mathbb{E}_{\mathcal{D}} [\mathbb{E}_{(\mathbf{x}, t)}[L]] &= \mathbb{E}_{\mathcal{D}} \left[\int \{y(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2 p(\mathbf{x}) d\mathbf{x} + \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt \right] \\ &= \int \mathbb{E}_{\mathcal{D}} [\{y(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2] p(\mathbf{x}) d\mathbf{x} + \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt \\ &= \int \{\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2 p(\mathbf{x}) d\mathbf{x} \\ &\quad + \int \mathbb{E}_{\mathcal{D}} [\{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]\}^2] p(\mathbf{x}) d\mathbf{x} + \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt. \end{aligned}$$

The Bias-Variance Decomposition

We obtain the following decomposition of the expected squared loss

$$\text{expected loss} = (\text{bias})^2 + \text{variance} + \text{noise},$$

where

$$(\text{bias})^2 = \int \{\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2 p(\mathbf{x}) d\mathbf{x},$$

$$\text{variance} = \int \mathbb{E}_{\mathcal{D}} \left[\{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]\}^2 \right] p(\mathbf{x}) d\mathbf{x},$$

$$\text{noise} = \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt.$$

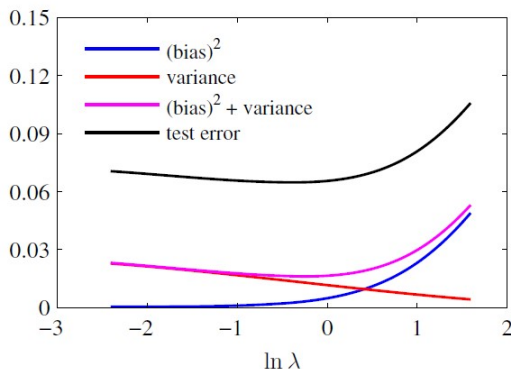
The Bias-Variance Decomposition

Given the regularized error

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \sum_{j=1}^M |w_j|^q,$$

- large λ : restrict the model's freedom for fitting data, leading to **high bias and low variance**
- small λ : warrant more freedom to fit data, resulting in **low bias and high variance**

The Bias-Variance Decomposition



Plot of squared bias and variance, together with their sum, as well as the test error, for a linear regression model with the sinusoidal basis function and the quadratic regularizer. The minimum value of $(\text{bias})^2 + \text{variance}$ occurs around $\ln \lambda = -0.31$, which is close to the value that gives the minimum error on the test data.

Logistic Regression

Logistic Regression for Binary Classification

Given the feature vector of an example $\mathbf{x}$, binary classification aims to predict its class label from the label candidate set $\{\mathcal{C}_1, \mathcal{C}_2\}$.

From the probabilistic view, the predicted class label corresponds the label $\mathcal{C}_k$ that maximizes the conditional probability $p(\mathcal{C}_k|\mathbf{x})$.

Suppose we can model class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$, as well as the class priors $p(\mathcal{C}_k)$, the posterior probabilities $p(\mathcal{C}_k|\mathbf{x})$ can be computed through Bayes' theorem.

$$\begin{aligned} p(\mathcal{C}_1|\mathbf{x}) &= \frac{p(\mathbf{x}|\mathcal{C}_1) p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1) p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2) p(\mathcal{C}_2)} \\ &= \frac{1}{1 + \exp(-a)} = \sigma(a), \end{aligned}$$

where we have defined $a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$ and $\sigma(a)$ is the *sigmoid* function defined by $\sigma(a) = \frac{1}{1+\exp(-a)}$.

Logistic Regression for Binary Classification

We can use a linear basis function model to parameterize the log of probability ratio $a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$, by setting $a = \mathbf{w}^T \phi(\mathbf{x})$ so that

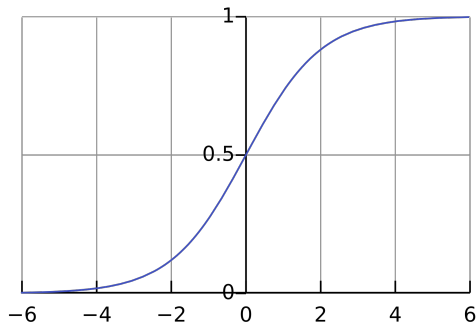
$$p(\mathcal{C}_1|\mathbf{x}) = \sigma(\mathbf{w}^T \phi(\mathbf{x})), \text{ and } p(\mathcal{C}_2|\mathbf{x}) = 1 - p(\mathcal{C}_1|\mathbf{x}).$$

Now, we get the *logistic regression* model.

Next, we use maximum likelihood to determine the parameters of the logistic regression model. To do this, we shall make use of the derivative of the sigmoid function, which can conveniently be expressed in terms of the sigmoid function

$$\frac{d\sigma}{da} = \sigma(1 - \sigma).$$

Logistic Regression for Binary Classification



Sigmoid Function $\sigma(a)$

Logistic Regression for Binary Classification

For a data set $\{\mathbf{x}_n, t_n\}$, where $t_n \in \{1, 0\}$, with $n = 1, \dots, N$, the likelihood function can be written as

$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N \{y_n(\mathbf{w})\}^{t_n} \{1 - y_n(\mathbf{w})\}^{1-t_n},$$

where $\mathbf{t} = (t_1, \dots, t_N)^T$ and $y_n(\mathbf{w}) = p(\mathcal{C}_1 = 1|\mathbf{x}_n) = \sigma(\mathbf{w}^T \phi(\mathbf{x}_n))$. We can define an error function by taking the negative logarithm of the likelihood, which gives the *cross-entropy* error function in the form

$$\begin{aligned} E(\mathbf{w}) &= -\ln p(\mathbf{t}|\mathbf{w}) \\ &= -\sum_{n=1}^N \{t_n \ln y_n(\mathbf{w}) + [1 - t_n] \ln [1 - y_n(\mathbf{w})]\}. \end{aligned}$$

To avoid overfitting, we can also add a regularization term to the *cross-entropy* error function.

Logistic Regression for Binary Classification

Taking the gradient of the error function with respect to $\mathbf{w}$, we obtain

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N \{y_n(\mathbf{w}) - t_n\} \phi_n(\mathbf{x}),$$

where we have made use of the equation $\frac{d\sigma}{da} = \sigma(1 - \sigma)$. We see that the contribution to the gradient from data point n is given by the 'error' $y_n(\mathbf{w}) - t_n$ between the target value and the prediction of the model, times the basis function vector $\phi_n(\mathbf{x})$.

Iterative Reweighted Least Squares

In the case of the linear regression models, whose sum-of-squares error function is a quadratic function of $\mathbf{w}$, we can obtain the closed-form optimal solution of $\mathbf{w}$ by setting the gradient to zero.

For logistic regression, there is no longer a closed-form solution, due to the nonlinearity of the logistic sigmoid function.

However, the departure from a quadratic form is not substantial. To be precise, the error function is convex, and hence has a unique minimum.

Furthermore, the error function can be minimized by an efficient iterative technique based on the Newton-Raphson iterative optimization scheme:

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \mathbf{H}(\mathbf{w}^{\text{old}})^{-1} \nabla E(\mathbf{w}^{\text{old}}),$$

where $\mathbf{H}(\mathbf{w}^{\text{old}})$ is the Hessian matrix at $\mathbf{w} = \mathbf{w}^{\text{old}}$, whose elements comprise the second-order derivatives of $E(\mathbf{w})$ with respect to the elements of $\mathbf{w}$.

Iterative Reweighted Least Squares

The gradient and Hessian matrix of *logistic regression's cross-entropy* error function are given by

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N \{y_n(\mathbf{w}) - t_n\} \phi(\mathbf{x}_n) = \Phi^T \{\mathbf{y}(\mathbf{w}) - \mathbf{t}\},$$

$$\mathbf{H} = \nabla \nabla E(\mathbf{w}) = \sum_{n=1}^N y_n(\mathbf{w}) \{1 - y_n(\mathbf{w})\} \phi(\mathbf{x}_n) \phi(\mathbf{x}_n)^T = \Phi^T \mathbf{R}(\mathbf{w}) \Phi,$$

where $\mathbf{y}(\mathbf{w}) = [y_1(\mathbf{w}), \dots, y_N(\mathbf{w})]^T$, $\Phi = [\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_N)]^T$ is an $N \times M$ matrix, and $\mathbf{R}(\mathbf{w})$ is an $N \times N$ diagonal matrix with entries

$$R_{nn}(\mathbf{w}) = y_n(\mathbf{w}) \{1 - y_n(\mathbf{w})\}.$$

Iterative Reweighted Least Squares

The Newton-Raphson update formula for the *logistic regression* model then becomes

$$\begin{aligned}\mathbf{w}^{\text{new}} &= \mathbf{w}^{\text{old}} - \left\{ \Phi^T \mathbf{R}(\mathbf{w}^{\text{old}}) \Phi \right\}^{-1} \Phi^T \left\{ \mathbf{y}(\mathbf{w}^{\text{old}}) - \mathbf{t} \right\} \\ &= \left\{ \Phi^T \mathbf{R}(\mathbf{w}^{\text{old}}) \Phi \right\}^{-1} \left\{ \Phi^T \mathbf{R}(\mathbf{w}^{\text{old}}) \Phi \mathbf{w}^{\text{old}} - \Phi^T \left\{ \mathbf{y}(\mathbf{w}^{\text{old}}) - \mathbf{t} \right\} \right\} \\ &= \left\{ \Phi^T \mathbf{R}(\mathbf{w}^{\text{old}}) \Phi \right\}^{-1} \Phi^T \mathbf{R}(\mathbf{w}^{\text{old}}) \mathbf{z}(\mathbf{w}^{\text{old}}),\end{aligned}$$

taking the form of a set of normal equations for a weighted least-squares problem, where $\mathbf{z}(\mathbf{w}^{\text{old}})$ is an N -dimensional vector with elements

$$\mathbf{z} = \Phi \mathbf{w}^{\text{old}} - \mathbf{R}(\mathbf{w}^{\text{old}})^{-1} \left\{ \mathbf{y}(\mathbf{w}^{\text{old}}) - \mathbf{t} \right\}.$$

Logistic Regression for Multi-Class Classification

For multi-class classification with label candidate set $\{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_K\}$, the posterior probabilities $p(\mathcal{C}_k|\mathbf{x})$ are given by a softmax transformation of linear functions of the feature variables

$$p(\mathcal{C}_k|\mathbf{x}) = y_k = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)},$$

where the ‘activations’ a_k are given by

$$a_k = \mathbf{w}_k^T \phi(\mathbf{x}).$$

We consider the use of maximum likelihood to determine the parameters $\{\mathbf{w}_k\}$ of this model. To do this, we will require the derivatives of y_k with respect to all of the activations a_j . These are given by

$$\frac{\partial y_k}{\partial a_j} = y_k (I_{kj} - y_j),$$

where I_{kj} are the elements of the $K \times K$ identity matrix, with $I_{kj} = 1$ for $k = j$ and $I_{kj} = 0$ for $k \neq j$.

Logistic Regression for Multi-Class Classification

Next we write down the likelihood function. This is most easily done using the 1-of- K coding scheme in which the target vector $\mathbf{t}_n$ for a feature vector $\phi(\mathbf{x}_n)$ belonging to class $\mathcal{C}_k$ is a binary vector with all elements zero except for element k , which equals one:

$$\mathbf{t}_n = (0, \dots, 1, \dots, 0)^T.$$

$\uparrow$
 k

The likelihood function is then given by

$$p(\mathbf{T}|\mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K p(\mathcal{C}_k|\mathbf{x}_n)^{t_{nk}} = \prod_{n=1}^N \prod_{k=1}^K y_{nk}^{t_{nk}},$$

where $y_{nk} = p(\mathcal{C}_k|\mathbf{x}_n)$, $\mathbf{T} = [\mathbf{t}_1, \dots, \mathbf{t}_N]^T$ is an $N \times K$ matrix target variables with elements t_{nk} .

Logistic Regression for Multi-Class Classification

Taking the negative logarithm of the likelihood function then gives

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = -\ln p(\mathbf{T}|\mathbf{w}_1, \dots, \mathbf{w}_K) = -\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk},$$

which is known as the *cross-entropy* error function for the multiclass classification problem.

We now take the gradient of the error function with respect to one of the parameter vectors $\mathbf{w}_j$:

$$\nabla_{\mathbf{w}_j} E(\mathbf{w}_1, \dots, \mathbf{w}_K) = \sum_{n=1}^N (y_{nj} - t_{nj}) \phi(\mathbf{x}_n).$$

Logistic Regression for Multi-Class Classification

We again appeal to the Newton-Raphson update to obtain the corresponding Iterative Reweighted Least Squares algorithm for the multiclass problem. This requires evaluation of the Hessian matrix that comprises blocks of size $M \times M$ in which block j, k is given by

$$\nabla_{\mathbf{w}_k} \nabla_{\mathbf{w}_j} E(\mathbf{w}_1, \dots, \mathbf{w}_K) = - \sum_{n=1}^N y_{nk} (I_{kj} - y_{nj}) \phi(\mathbf{x}_n) \phi(\mathbf{x}_n)^T.$$

As with the two-class problem, the Hessian matrix for the multiclass logistic regression model is positive definite and so the error function again has a unique minimum.

Christopher M. Bishop. **Pattern Recognition and Machine Learning**.
New York: Springer; 2006 Aug 17.

- Linear Regression: Section 3.1-3.2
- Logistic Regression: Section 4.3

The End